JC2001: Introduction to Software Engineering

Lecture 7: Structural & Behavioural Models

Lecture Schedule: Week 6

Dr Zhenhua Yu zhenhua.yu@abdn.ac.uk

Dr Riazul Islam riazul.islam@abdn.ac.uk

Dr Shahzad Mumtaz shahzad.mumtaz@abdn.ac.ukc

Learning objectives

- Understand the fundamental system modeling perspectives of structure, and behaviour.
- Understand the diagram types in the Unified Modeling Language (UML) and how these diagrams can be used in system modeling.

Reference: Ian Sommerville, Software Engineering – Chapter 5, 10th edition, Pearson, 2015.

Outline of lecture

- Introduction to structural models.
- Introduction to behavioural models.

Structural models

- Structural models of software display the organisation of a system in terms of the **components** that make up that system and their **relationships**.
- Structural models may be
  - **Static models**, which show the organisation of the system design, or
  - **Dynamic models**, which show the organisation of the system when it is executing.
- These are not the same thing – the dynamic organisation of a system as a set of interacting threads may be very different from a static model of the system components.

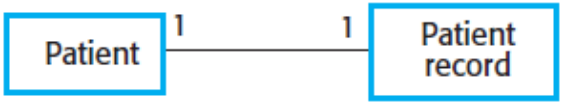
Class diagrams

- Class diagrams are used when developing an object-oriented system model to **show the classes** in a system **and the associations** between these classes.
    - Informally, an object class can be thought of as a general definition of one kind of system object.
    - An association is a link between classes indicating some relationship exists between these classes.

- When developing models during the early stages of the software engineering process, objects represent something in the real world, such as a patient, a prescription, or a doctor.

- As an implementation is developed, you define implementation objects to represent data that is manipulated by the system.

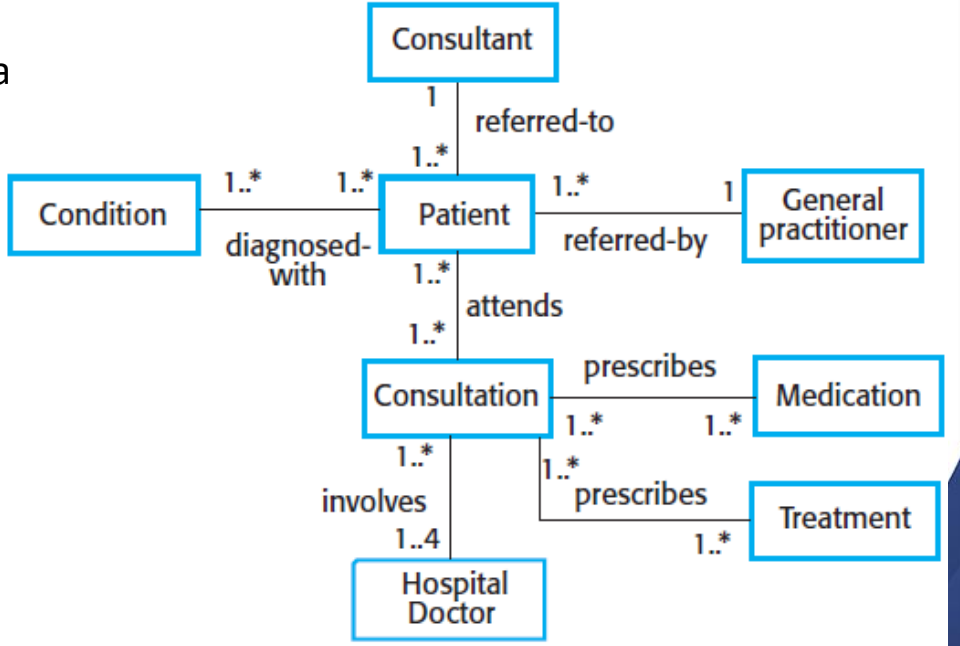
Mentcare: Class diagram



- Patient has one record, which indicates a 1 to 1 relationship between the Patient object and the Patient record object. Here, each patient has exactly one record, and each record maintains information about exactly one patient.

Mentcare: Class diagram

- In this class diagram, the objects of class Patient are also involved in relationships with a number of other classes, such as Consultant, Condition, General practitioner, Consultation, Medication, Hospital Doctor, and Treatment.
- Here, an indefinite number of objects of a class are involved in the association. For example, the 1..* multiplicity between Patient and Condition shows that a patient may suffer from several conditions, and that the same condition may be associated with several patients.



Mentcare: Class diagram

- At this level of detail, class diagrams look like semantic data models.
- Semantic data models are used in database design (we will cover this in Principles and Practices of Database Systems). They show the data entities, their associated attributes, and the relations between these entities.
- However, UML does not include a diagram type for database modeling, as it models data using objects and their relations.

Case study: Sim City game

SimCity is an open-ended city building video game. *SimCity* titles are real-time management and construction simulators. Across most titles, the player (acting as mayor) is given a blank map to begin and must expand the city with the budget provided. As the city matures, the player may be able to add government and other special buildings (such as a mayor's house or courthouse), depending on how large the city is. Proper management of the city requires citizens to be provided with basic utilities (electricity, water and sometimes waste management) along with public services such as health, education, safety, parks and leisure facilities. The primary source of income is taxation. The player may make deals with neighbouring cities to sell or buy services, as long as a connection is made to the neighbour for that service, such as electricity cables.

Draw a Class diagram for the Sim City game

- **Step 1**: Identify objects from the description. How? Look for nouns in the description.
  - A **noun** is a word that represents a person, thing, concept or place, e.g., John, house, river.
  - Nouns are often preceded by an article (“the”, “a”, “an”).

- **Mock quiz 1**: Identify all objects from the Sim City game description.

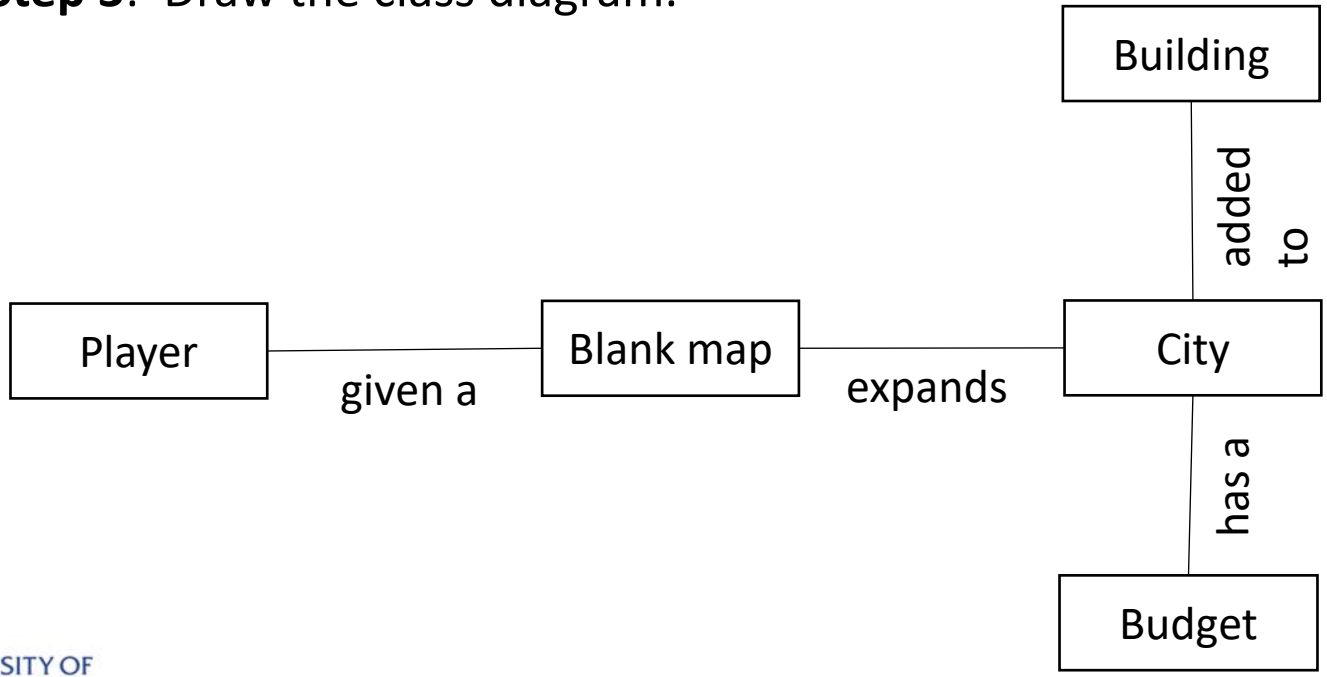
Draw a Class diagram for the Sim City game

- **Step 2**: Identify relationships from the description. How? Look for verbs in the sentence.
  - A **verb** is a word used to describe an action, state or occurrence.
  - For example, “Jennifer walked to the store”. In this sentence, walked is the verb that shows an action.

- **Mock quiz 2**: Identify all verbs from the Sim City game description.

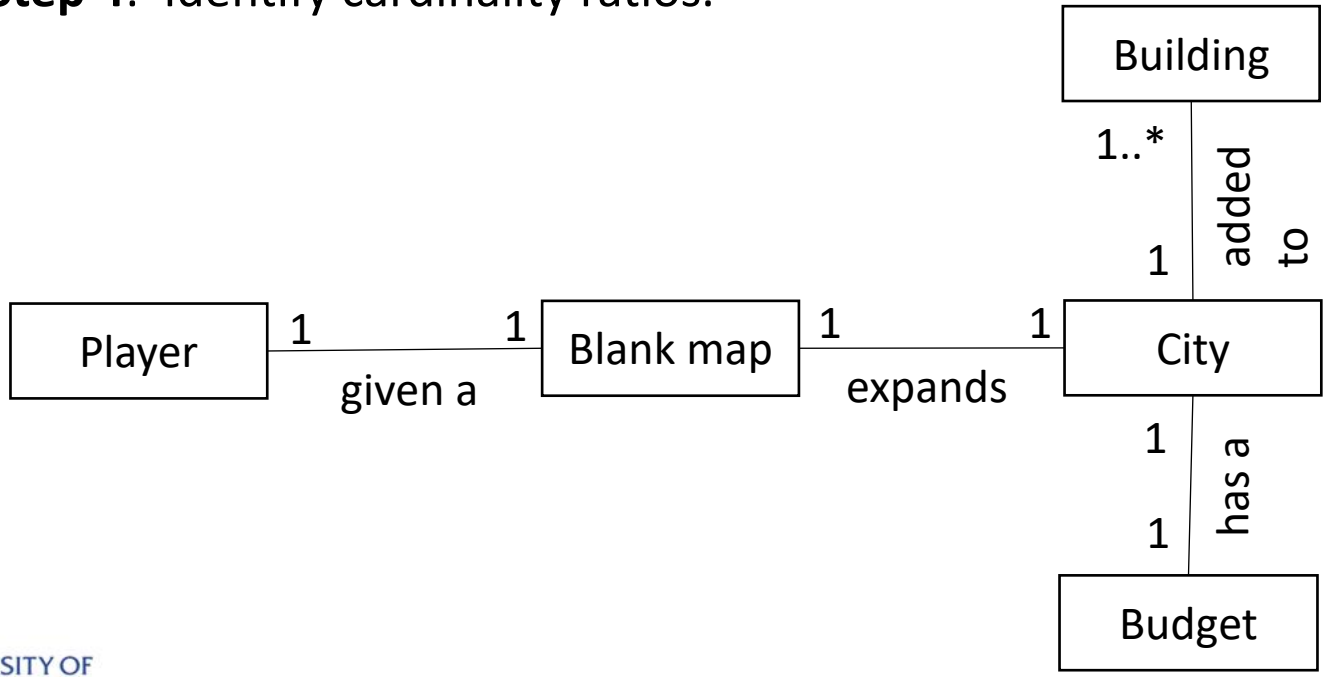
Draw a Class diagram for the Sim City game

- **Step 3**: Draw the class diagram.



Draw a Class diagram for the Sim City game

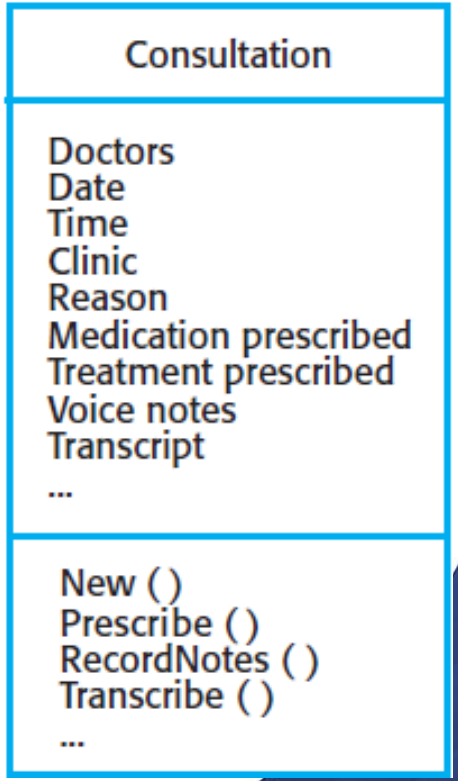
- **Step 4**: Identify cardinality ratios.



Mentcare: Class structure

• In UML, you show attributes and operations by extending the simple rectangle that represents a class. An class representing a consultation between a doctor and a patient has the following:

- The **name of the class** is in the top section.
- The **class attributes** are in the middle section. This includes the attribute names, and optionally, their types.
- The **operations** (called methods in Java, C++ and other object-oriented programming languages) associated with the object class are in the lower section of the rectangle.



Mock quiz

- Specify the class structure for the Player class.

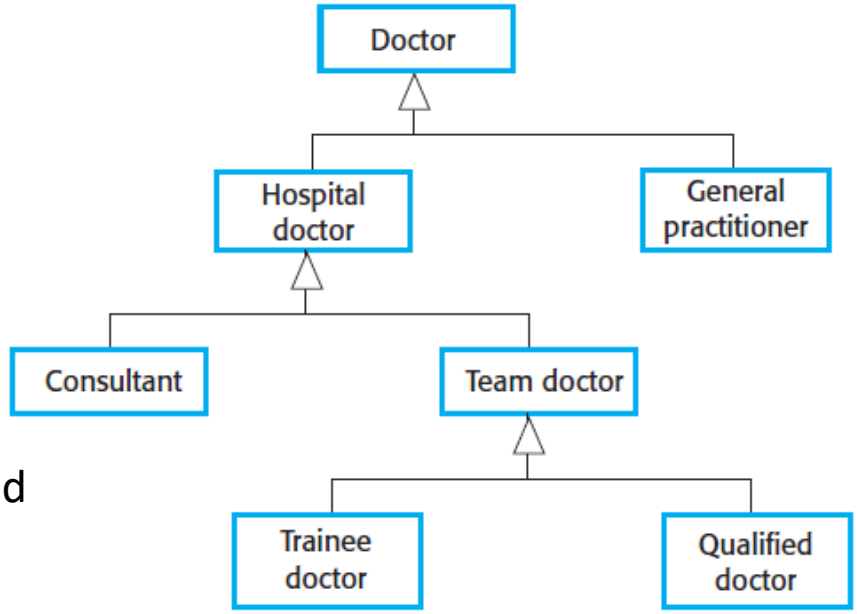
Generalisation

- Generalisation is an everyday technique that we use to manage complexity. Rather than learn the detailed characteristics of everything that we experience, we learn about **general classes** and learn the **characteristics of these classes**.

- When modeling systems, it is useful to examine the classes in a system to see if there is scope for **generalisation** and **class creation**.
    - This means that common information will be maintained in one place only.
    - This is good design practice as it means that, if changes are proposed, then you do not have to look at all classes in the system to see if they are affected by the change.

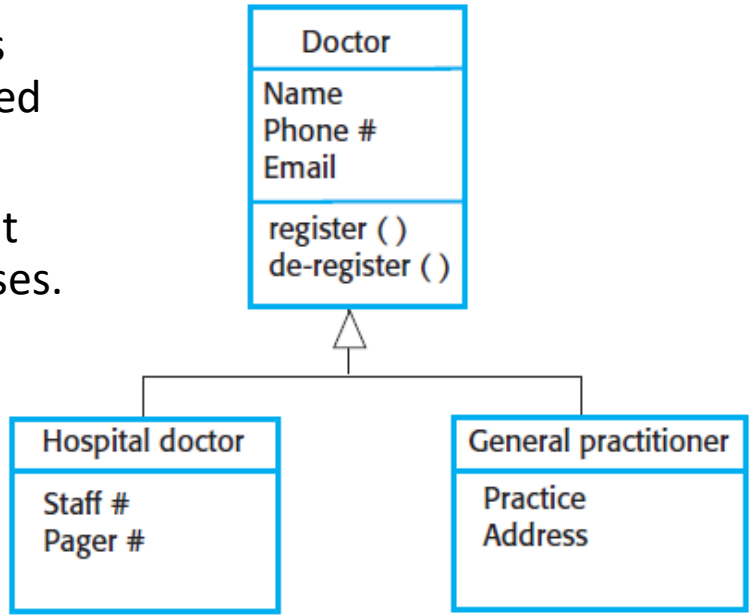
Generalisation

- UML has a specific type of association to denote generalisation. The generalisation is shown as an **arrowhead pointing up** to the more general class.
    - Hospital doctors and general practitioners can be generalised as doctors.
    - There are three types of hospital doctor: trainee doctor, consultant, and qualified doctor.



Generalisation

- In a generalisation, the attributes and operations associated with high-level classes are also associated with the lower-level classes.
- The lower-level classes are subclasses that inherit the attributes and operations from their superclasses.
- These lower-level classes then add more specific attributes and operations.



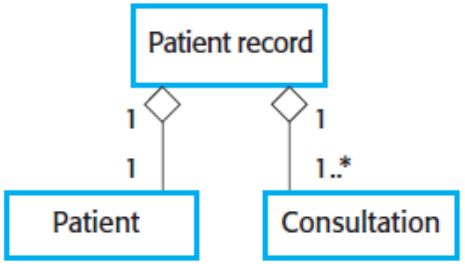
Mock quiz

- In the Sim City game, buildings are constructed by the government or private contractors. Buildings that can be accessed by the public include libraries, hotels, houses, universities, schools, hospitals and police stations. Building that can only be accessed by authorised personnel include casinos, military bases and fire stations. Draw a generalisation class diagram for the Building class.

Aggregation

- Objects in the real world are often made up of different parts. The UML provides a special type of association between classes called *aggregation*. It means that one object (the whole) is composed of other objects (the parts). To define aggregation, a diamond shape is added to the link next to the class that represents the whole.

- A Patient record is an aggregate of Patient and an indefinite number of Consultations.
- This record maintains personal patient information as well as individual record for each consultation with a doctor.



Behavioural models

- Behavioural models are models of the **dynamic behaviour of a system** as it is executing. They show what happens or what is supposed to happen when a system responds to a stimulus from its environment. The stimuli may be either data or events:
    - Data becomes available that has to be processed by the system. The availability of the data triggers the processing.
    - An event happens that trigger system processing. Events may have associated data, although this is not always the case.

Behavioural models

- Many business systems are data-processing systems that are primarily driven by data. They are controlled by the data input to the system, with relatively little external event processing. For example, a phone billing system will accept information about calls made by a customer, calculate the costs of these calls, and generate a bill for that customer.

- In contrast, real-time systems are usually event-driven, with limited data processing. For example, a landline phone switching system responds to events such as ``handset activated'' by generating a dial tone, pressing keys on a handset by capturing the phone number, etc.

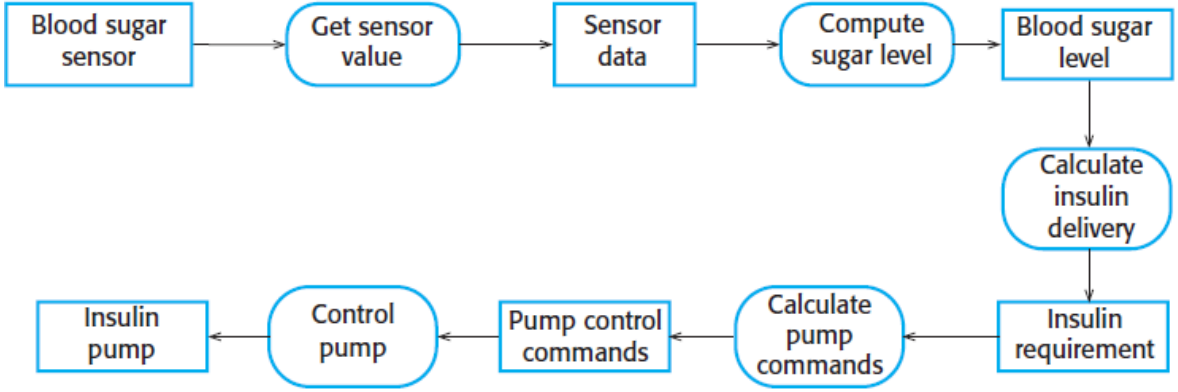
Data-driven modeling

- Data-driven modeling show the *sequence of actions* involved in *processing input data* and generating an associated output. These models can be used during the analysis of requirements as they show end-to-end processing in a system.

- Data flow models are useful because tracking and documenting how data associated with a particular process moves through the system help analysts and designers understand what is going on in the process. Data flow diagrams (DFD) are simple and intuitive, and so are more accessible to stakeholders than some other types of model.

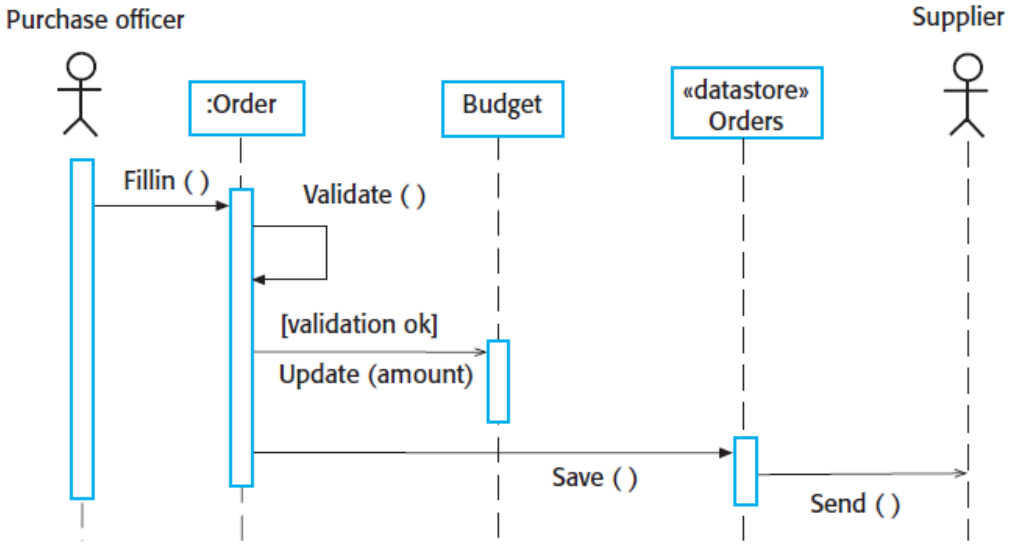
Data flow diagram: Insulin pump's operation

- DFD shows the chain of processing involved in the insulin pump's software.
- Processing steps are represented as activities (round rectangles).
- Data flowing between these activities are represented as objects (rectangles).



Sequence diagram: Order processing

- An alternative way of showing the sequence of processing in a system is to use UML sequence diagram.
- A sequence model of processing an order and sending it to a supplier is shown on the right.
- Sequence models highlight objects in a system. In contrast, DFDs highlight operations or activities.



Non-experts find DFDs more intuitive, but engineers prefer sequence diagrams.

Lessons learned

- We gave details on structural and behavioural models.
- We presented examples and showed how to construct these models.